

Diseño e implementación de una herramienta web para el análisis de imágenes hiperespectrales

Marc Garcia Martín

Resum—Resum del projecte, màxim 10 línies.

.....

.....

.....

.....

.....

.....

.....

.....

.....

Paraules clau—Paraules clau del projecte, màxim 2 línies.

.....

Abstract—Versió en anglès del resum.

.....

.....

.....

.....

.....

.....

.....

.....

.....

Index Terms—Versió en anglès de les paraules clau.

.....

1 INTRODUCCIÓN

Las imágenes hiperespectrales permiten analizar materiales y superficies a partir de su firma espectral, capturando información en cientos de bandas del espectro electromagnético. A diferencia de las imágenes convencionales RGB (que contienen únicamente tres canales), cada píxel en una imagen hiperespectral puede interpretarse como un vector de datos que describe propiedades físicas y químicas del objeto observado.

cámara hiperespectral puede emplearse para detectar alimentos y cosechas en mal estado antes de su distribución comercial, clasificándolas automáticamente según su firma espectral. Este ejemplo permite comprender el potencial de la tecnología, aunque la herramienta desarrollada en este proyecto estará diseñada para ser genérica y permitir un análisis y clasificación de cualquier tipo de imagen hiperespectral.

Un caso más cotidiano puede observarse en productos alimentarios adquiridos en un supermercado, como Mercadona. Por ejemplo, una manzana puede presentar un aspecto visual aparentemente correcto en el momento de la compra, pero deteriorarse rápidamente a los pocos días debido a daños internos producidos durante su transporte o manipulación. Estos defectos, que a simple

- E-mail de contacto: marcgarciamartin37@gmail.com
- Menció realitzada: Ingenieria del Software
- Trabajo tutorizado por: Coen Antens
- Curso 2025/26

Como ejemplo ilustrativo, en el ámbito agrícola una

vista resultan imperceptibles, pueden deberse a pequeños golpes que alteran la estructura interna del fruto. Mediante el uso de cámaras hiperespectrales capaces de analizar la información espectral del producto, sería posible detectar este tipo de daños internos antes de que el alimento llegue al consumidor, mejorando así los procesos de control de calidad en la cadena de distribución.

En este contexto, este Trabajo de Fin de Grado propone el desarrollo de una aplicación web abierta, accesible y orientada tanto a la experimentación como a la docencia, que permita visualizar, explorar, editar y analizar imágenes hiperespectrales mediante una herramienta versátil y aplicando técnicas de Machine Learning y Deep Learning.

2 OBJETIVOS

En este apartado se definen los objetivos del proyecto, estableciendo el propósito principal que se pretende alcanzar con el desarrollo de la herramienta. A partir de este objetivo general se derivan una serie de objetivos específicos que permiten estructurar el trabajo y guiar el desarrollo del sistema durante las distintas fases del proyecto.

2.1 Objetivo General

Desarrollar una aplicación web interactiva que permita la visualización, exploración y clasificación de imágenes hiperespectrales mediante algoritmos de aprendizaje automático, proporcionando una herramienta accesible, modular y extensible para el análisis científico y educativo.

2.2 Objetivos Específicos

- Realizar una revisión del estado del arte en procesamiento y clasificación de imágenes hiperespectrales para poder recoger los requisitos necesarios para el desarrollo.
- Diseñar una arquitectura software escalable basada en backend con Python con API REST, frontend interactivo desarrollado en Flutter Web, y un sistema modular para integración de modelos de Machine Learning.
- Implementar funcionalidades de visualización que permitan la carga de imágenes hiperespectrales en distintos formatos.
- Implementar modelos de clasificación y evaluar su rendimiento mediante métricas objetivas.

3 METODOLOGÍA

La metodología utilizada para el desarrollo del proyecto es de tipo Agile con un modelo Scrum adaptado a un entorno de desarrollo individual. Este enfoque permite organizar el trabajo mediante iteraciones cortas y realizar

un seguimiento continuo del progreso del proyecto. Durante el desarrollo se realiza un seguimiento continuo de las tareas, revisando periódicamente el estado del proyecto y ajustando la planificación en función de los avances obtenidos.

3.1 Herramienta de seguimiento

Para realizar el seguimiento del desarrollo del proyecto se utiliza la herramienta Jira [10], que permite gestionar las tareas, organizar los sprints y mantener un control detallado del progreso del proyecto. Las tareas se agrupan dentro de sprints, que tienen una duración aproximada de cuatro a cinco semanas. Cada sprint incluye un conjunto de tareas concretas cuyo objetivo es implementar nuevas funcionalidades o mejorar las existentes. Este enfoque permite realizar entregas progresivas del proyecto y facilita la identificación temprana de posibles problemas durante el desarrollo.

3.2 Herramientas de desarrollo

Para la implementación de la herramienta se utilizan diferentes tecnologías orientadas tanto al procesamiento de datos como al desarrollo de aplicaciones web. El backend de la aplicación se desarrolla en Python, utilizando bibliotecas de aprendizaje automático como scikit-learn [6], que permiten implementar algoritmos de clasificación y análisis de datos.

En cuanto a la interfaz de usuario, se emplea Flutter [11], una herramienta de desarrollo multiplataforma que permite crear aplicaciones web modernas con una interfaz interactiva y accesible desde cualquier navegador.

El uso conjunto de estas tecnologías permite desarrollar una herramienta flexible y modular, donde el procesamiento de las imágenes hiperespectrales se realiza en el backend mientras que la visualización y la interacción con el usuario se gestionan desde la interfaz web.

4 PLANIFICACIÓN

La planificación del proyecto se realiza con el objetivo de organizar las diferentes tareas necesarias para el desarrollo de la herramienta y establecer una distribución temporal adecuada de las mismas. Para ello se identifican las principales fases del proyecto, desde el análisis inicial y el diseño del sistema hasta la implementación, las pruebas y la documentación final.

Siguiendo la metodología de desarrollo descrita en el apartado anterior, el trabajo se estructura en cuatro sprints de varias semanas de duración. En cada uno de estos sprints se agrupan diferentes tareas relacionadas con una fase concreta del desarrollo, permitiendo avanzar de forma progresiva en la implementación del sistema y realizar un seguimiento continuo del progreso del proyecto.

Para facilitar la organización y el seguimiento de las tareas se utiliza Jira [10], donde se definen las diferentes actividades del proyecto y su duración estimada,

divididas en cinco fases: Diseño, Programación, Modelo, Pruebas y Documentación. A partir de esta información se genera un diagrama de Gantt que permite visualizar de forma clara la planificación temporal del proyecto y la relación entre las distintas tareas.

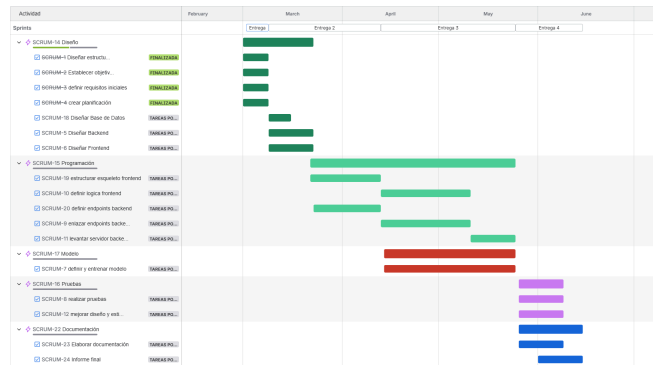


Figura 1: Diagrama de Gantt

5 ESTADO DEL ARTE

El desarrollo de herramientas para el análisis de imágenes hiperespectrales ha crecido significativamente en los últimos años debido al aumento de aplicaciones en ámbitos científicos e industriales. Actualmente existen soluciones comerciales especializadas, como las desarrolladas por Clyde HSI [1] y PerClass [2].

Sin embargo, la mayoría de estas soluciones están orientadas a entornos industriales y suelen ser herramientas propietarias, con acceso restringido o licencias comerciales. Por este motivo, su utilización en contextos académicos o educativos puede resultar limitada.

A pesar del creciente número de aplicaciones, actualmente existen pocas herramientas accesibles que permitan visualizar y experimentar con datos hiperespectrales de forma sencilla desde una interfaz web. En muchos casos, el análisis requiere el uso de software especializado o conocimientos avanzados en programación científica.

En este contexto, el presente proyecto propone el desarrollo de una herramienta web que permita cargar, visualizar y analizar imágenes hiperespectrales de forma interactiva, integrando algoritmos de aprendizaje automático en un entorno accesible y orientado tanto a la investigación como al aprendizaje.

5.1: Hardware: Cámaras Hiperespectrales

El análisis de imágenes hiperespectrales requiere dispositivos de captura capaces de registrar información espectral en múltiples longitudes de onda. Actualmente existen diversas cámaras hiperespectrales en el mercado que permiten capturar este tipo de datos con alta resolución espectral.

Entre los fabricantes más conocidos se encuentran

empresas como Specim [3], que desarrolla cámaras hiperespectrales utilizadas en aplicaciones industriales, agrícolas y medioambientales. Otros fabricantes relevantes incluyen a Headwall Photonics [4], cuyos sistemas se emplean en investigación científica y monitorización ambiental, y Cubert GmbH [5], que produce cámaras compactas capaces de capturar información espectral en una única adquisición.

5.2: Software para el análisis de las imágenes

El procesamiento y análisis de imágenes hiperespectrales requiere herramientas software capaces de gestionar grandes volúmenes de datos y aplicar técnicas de análisis avanzado.

En el ámbito científico es habitual utilizar bibliotecas de programación como scikit-learn [6], una librería de aprendizaje automático para Python que proporciona algoritmos de clasificación, regresión y clustering ampliamente utilizados en análisis de datos. Estas técnicas permiten clasificar píxeles o regiones de una imagen hiperespectral según sus características espectrales.

Otra herramienta ampliamente utilizada en el procesamiento de imágenes científicas es ImageJ [7], un software de código abierto desarrollado originalmente por el National Institutes of Health (NIH) que permite visualizar, procesar y analizar imágenes multidimensionales mediante diferentes plugins y extensiones.

5.3 Tecnologías web para el desarrollo de la app

El desarrollo de aplicaciones web modernas permite crear herramientas accesibles desde cualquier navegador sin necesidad de instalar software especializado. Para ello es habitual utilizar frameworks que faciliten la creación de aplicaciones web y la integración con sistemas de procesamiento de datos.

En el ecosistema de Python destacan frameworks como Flask [8] y Django [9], que permiten desarrollar aplicaciones web y APIs REST capaces de comunicarse con módulos de procesamiento de datos y aprendizaje automático.

5.4 Datasets y repositorios de imágenes hiperespectrales

El desarrollo de algoritmos de análisis y clasificación requiere el uso de conjuntos de datos adecuados. En el ámbito hiperespectral existen diversos datasets públicos utilizados en investigación. Algunos de los más conocidos son:

- Indian Pines [12]: dataset ampliamente utilizado en clasificación de cultivos.
- Pavia University [13]: imagen urbana utilizada en tareas de clasificación supervisada.

- Salinas [14]: dataset agrícola con alta resolución espacial.

Estos conjuntos de datos actúan como repositorios de referencia para la evaluación de algoritmos, aunque no constituyen un data warehouse en sentido estricto.

Actualmente no existe un estándar universal de data warehouse para imágenes hiperespectrales, ya que los datos suelen estar distribuidos en repositorios de investigación o instituciones específicas. Sin embargo, iniciativas como la NASA [15] o la ESA [16] proporcionan acceso a grandes volúmenes de datos espectrales a través de sus plataformas de observación terrestre.

6 DESARROLLO

En este apartado se describe el proceso de desarrollo de la aplicación web propuesta, incluyendo tanto la implementación del backend como del frontend, así como la integración de los distintos componentes del sistema. El objetivo principal es construir una herramienta capaz de procesar, visualizar y analizar imágenes hiperespectrales mediante técnicas de aprendizaje automático, ofreciendo una interfaz accesible desde el navegador.

El desarrollo se lleva a cabo siguiendo una arquitectura cliente-servidor, en la que el procesamiento de datos se realiza en el backend, mientras que la interacción con el usuario se gestiona desde el frontend.

6.1 Arquitectura del sistema

La aplicación se diseña siguiendo una arquitectura basada en servicios, separando claramente las responsabilidades entre frontend y backend, como se puede ver en la figura 2.

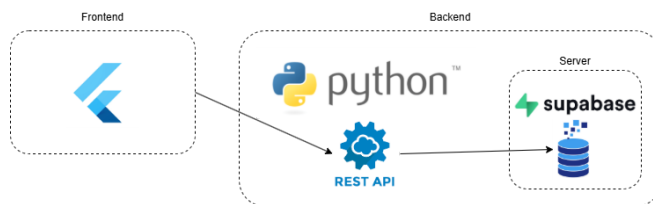


Figura 2: Arquitectura del sistema

Por un lado, el backend, desarrollado en Python, se encarga del procesamiento de las imágenes hiperespectrales, la ejecución de los algoritmos de aprendizaje automático y la gestión de los datos. Por otro lado, el frontend, desarrollado con Flutter, proporciona una interfaz interactiva que permite al usuario cargar imágenes, visualizar resultados y configurar los parámetros del análisis.

La comunicación entre ambos componentes se realiza mediante una API REST, lo que permite desacoplar el sistema y facilitar su escalabilidad y mantenimiento.

6.2 Desarrollo del backend

El backend constituye el núcleo funcional de la aplicación, siendo responsable del procesamiento de las imágenes hiperespectrales y de la ejecución de los modelos de clasificación.

Para la implementación se han utilizado librerías del ecosistema de Python como NumPy para el manejo de datos, SciPy para la carga de datasets y scikit-learn para la implementación de algoritmos de aprendizaje automático.

El flujo de procesamiento desarrollado incluye las siguientes etapas:

- Carga de imágenes hiperespectrales desde archivos externos.
- Transformación de la imagen en un conjunto de datos estructurado.
- Preprocesado de los datos, eliminando valores no relevantes.
- Entrenamiento de modelos de clasificación supervisada.
- Evaluación del rendimiento del modelo.
- Aplicación del modelo sobre la imagen completa.

Como resultado de este proceso, se obtiene una clasificación de cada píxel en función de su firma espectral.

6.3 Procesamiento y clasificación de imágenes hiperespectrales

Para validar el funcionamiento del sistema, se ha utilizado el dataset Indian Pines [12], del cual podemos observar su *groundtruth* en la figura 3, ampliamente reconocido en el ámbito de la teledetección.



Figura 3: "Groundtruth" del dataset Indian Pines

Cada píxel de la imagen se representa mediante un vector que contiene información de múltiples bandas espectrales. A partir de estos datos, se entrena un modelo de clasificación basado en Random Forest, capaz de identificar patrones en las firmas espectrales y asignar una clase a cada píxel.

Una vez entrenado el modelo, este se aplica sobre la totalidad de la imagen, generando un mapa de

clasificación que representa la distribución espacial de las distintas clases presentes en la escena.

Este proceso demuestra la capacidad del sistema para analizar imágenes hiperespectrales y extraer información relevante de forma automática.

6.4 Visualización de resultados

La visualización de los resultados se realiza mediante la generación de mapas de clasificación, en los que cada píxel se representa mediante un color asociado a la clase asignada por el modelo.

Para ello se utiliza la librería matplotlib, que permite representar de forma gráfica la distribución espacial de las clases. Esta representación facilita la interpretación de los resultados y permite identificar patrones visuales en la imagen, como se observa en la figura 4.

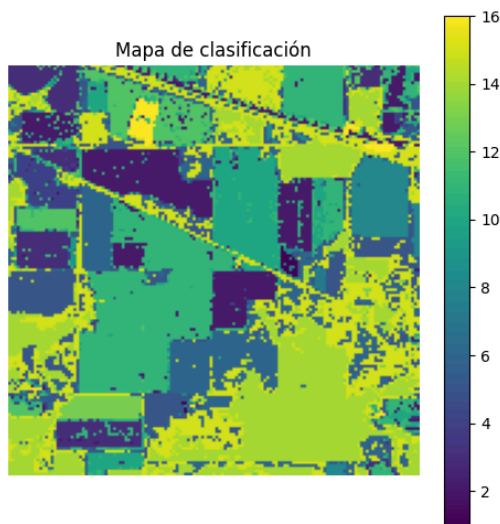


Figura 4: Resultado de clasificación del dataset Indian Pines

0 CONCLUSIÓN

.....

.....

.....

.....

USO DE LA IA GENERATIVA

.....

.....

.....

.....

AGRADECIMIENTOS

.....

.....

.....

.....

BIBLIOGRAFÍA

[1] ClydeHSI, "Hyperspectral Imaging Solutions." [En línea].

Disponible: <https://www.clydehsi.com>. [Accedido: 22-feb-2026].

[2] PerClass BV, "Hyperspectral Image Classification Software." [En línea]. Disponible: <https://www.perclass.com>. [Accedido: 22-feb-2026].

[3] Specim, Spectral Imaging Ltd., "Hyperspectral Cameras and Systems." [En línea]. Disponible: <https://www.specim.com>. [Accedido: 22-feb-2026].

[4] Headwall Photonics Inc., "Hyperspectral Imaging Systems." [En línea]. Disponible: <https://www.headwallphotonics.com>. [Accedido: 22-feb-2026].

[5] Cubert GmbH, "Snapshot Hyperspectral Cameras." [En línea]. Disponible: <https://www.cubert-gmbh.com>. [Accedido: 22-feb-2026].

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python." [En línea]. Disponible: <https://scikit-learn.org>. [Accedido: 23-feb-2026].

[7] National Institutes of Health, "ImageJ — Image Processing Program." [En línea]. Disponible: <https://imagej.nih.gov/ij/>. [Accedido: 23-feb-2026].

[8] A. Ronacher, "Flask Web Framework." [En línea]. Disponible: <https://flask.palletsprojects.com>. [Accedido: 23-feb-2026].

[9] Django Software Foundation, "Django Web Framework." [En línea]. Disponible: <https://www.djangoproject.com>. [Accedido: 23-feb-2026].

[10] Atlassian, "Jira — Issue Tracking and Project Management Tool." [En línea]. Disponible: <https://www.atlassian.com/software/jira>. [Accedido: 23-feb-2026].

[11] Google, "Flutter — UI Toolkit for Building Applications." [En línea]. Disponible: <https://flutter.dev>. [Accedido: 23-feb-2026].

[12] Purdue University, "Indian Pines Dataset." [En línea]. Disponible: https://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes. [Accedido: 29-mar-2026].

[13] University of Pavia, "Pavia University Dataset." [En línea]. Disponible: https://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes. [Accedido: 29-mar-2026].

[14] University of California, "Salinas Dataset." [En línea]. Disponible: https://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes. [Accedido: 29-mar-2026].

[15] NASA, "Earth Data." [En línea]. Disponible: <https://earthdata.nasa.gov>. [Accedido: 29-mar-2026].

[16] European Space Agency, "Copernicus Programme." [En línea]. Disponible: <https://www.esa.int>. [Accedido: 29-mar-2026].

APÉNDICE

A1. REQUISITOS

ID	Requisito	Descripción	Prioridad
RF1	Carga de imágenes hiperespectrales	La aplicación permite al usuario cargar imágenes hiperespectrales desde su dispositivo.	Alta
RF2	Validación de formato	El sistema verifica que el formato del archivo cargado es compatible.	Alta
RF3	Lectura de metadatos	El sistema extrae y muestra información básica de la imagen (dimensiones, número de bandas, etc.).	Media
RF4	Visualización inicial de la imagen	El sistema muestra una representación inicial de la imagen cargada.	Alta
RF5	Selección de banda espectral	El usuario puede seleccionar una banda concreta para su visualización.	Alta
RF6	Visualización de múltiples bandas	El sistema permite combinar varias bandas para generar imágenes compuestas.	Media
RF7	Navegación por la imagen	El usuario puede hacer zoom y desplazarse por la imagen.	Alta
RF8	Selección de píxeles	El usuario puede seleccionar un píxel específico de la imagen.	Alta
RF9	Selección de regiones (ROI)	El usuario puede seleccionar una región de interés dentro de la imagen.	Media
RF10	Visualización de firma espectral	El sistema muestra el vector espectral asociado a un píxel o región seleccionada.	Alta
RF11	Exportación de datos espectrales	El usuario puede exportar los datos espectrales seleccionados.	Alta
RF12	Preprocesamiento de datos	El sistema permite aplicar operaciones básicas (normalización, filtrado, etc.).	Media
RF13	Aplicación del modelo	El sistema aplica el modelo de clasificación entrenado a la imagen completa.	Alta
RF14	Visualización de resultados de clasificación	El sistema muestra el resultado de la clasificación como mapa de clases.	Alta
RF15	Interacción con clases	El usuario puede seleccionar una clase y visualizar sus píxeles asociados.	Alta
RF16	Guardado de resultados	El sistema permite guardar los resultados obtenidos.	Media

ID	Requisito	Descripción	Prioridad
RF17	Carga de resultados previos	El usuario puede cargar resultados previamente guardados.	Media
RF18	Comunicación frontend-backend	El sistema permite la comunicación mediante API REST.	Alta
RF19	Gestión de errores	El sistema informa al usuario en caso de errores en carga o procesamiento.	Alta
RF20	Reinicio de análisis	El usuario puede reiniciar el proceso de análisis con una nueva imagen.	Alta
RF21	Registro del usuario	El sistema debe permitir al usuario registrar su cuenta mediante correo y contraseña.	Alta
RF22	Inicio de sesión del usuario	El sistema debe permitir al usuario iniciar sesión en su cuenta mediante correo y contraseña.	Alta
RNF1	Usabilidad	La aplicación debe ser intuitiva y fácil de usar, con un enfoque accesible para usuarios sin experiencia avanzada.	Alta
RNF2	Rendimiento	El sistema debe procesar y visualizar imágenes en tiempos razonables, incluso con datos de gran tamaño.	Alta
RNF3	Portabilidad	La aplicación debe ser accesible desde un navegador web sin necesidad de instalación adicional.	Alta
RNF4	Mantenibilidad	El código debe estar estructurado de forma modular para facilitar su mantenimiento y evolución.	Media

A2. DIAGRAMAS DE CASO DE USO

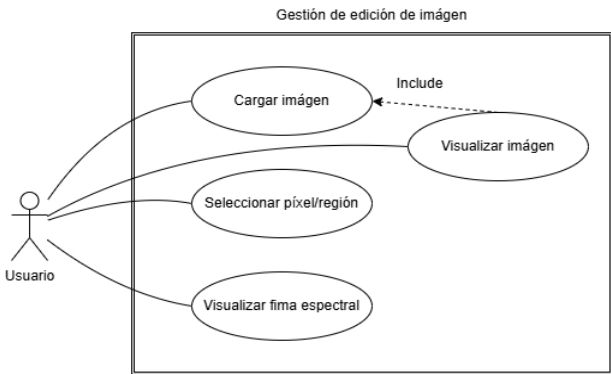


Diagrama 1: Gestión de edición de imagen

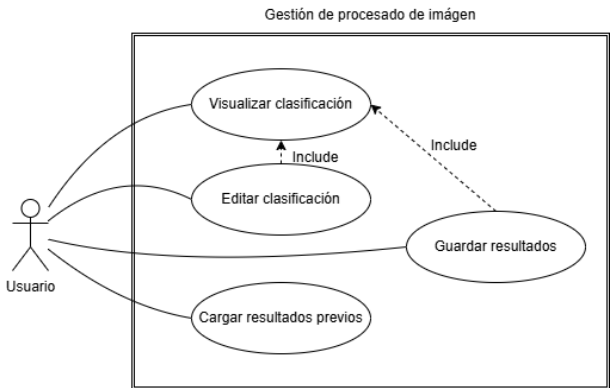


Diagrama 2: Gestión de procesado de imagen

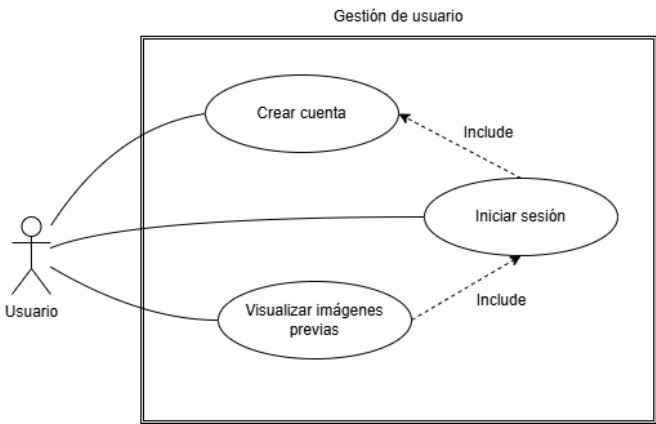


Diagrama 3: Gestión de usuario